

OntoSage 2.0: System Architecture & Technical Report

Enabling Zero-Knowledge Human–Building Interaction through Agentic AI

Author: Suhas Devmane

Institution: Cardiff University

Date: February 2026

Version: 2.0.0

License: MIT

Repository: github.com/suhasdevmane/OntoBot

Table of Contents

1. [Executive Summary](#)
2. [Problem Statement & Motivation](#)
3. [System Overview](#)
4. [Architecture](#)
 - [4.1 High-Level Architecture](#)
 - [4.2 Service Topology](#)
 - [4.3 Network Architecture](#)
5. [Core Components](#)
 - [5.1 Orchestrator Service](#)
 - [5.2 Multi-Agent System](#)
 - [5.3 RAG Service](#)
 - [5.4 Code Executor Service](#)
 - [5.5 Frontend Application](#)
6. [Data Layer](#)
 - [6.1 GraphDB \(Knowledge Graph\)](#)
 - [6.2 MySQL \(Time-Series Sensor Data\)](#)
 - [6.3 Redis \(State & Cache\)](#)
 - [6.4 PostgreSQL \(User Data\)](#)
 - [6.5 Sensor Map Cache](#)
7. [Workflow: End-to-End Query Resolution](#)
 - [7.1 LangGraph State Machine](#)
 - [7.2 Intent Classification & Routing](#)
 - [7.3 Query Resolution Flows](#)
 - [7.4 Clarification & Sensor Discovery](#)
8. [LLM Integration & Provider Flexibility](#)
9. [Authentication & Security](#)
10. [Monitoring & Observability](#)
11. [Deployment](#)
 - [11.1 Prerequisites & Hardware Requirements](#)
 - [11.2 Step-by-Step Setup](#)

- 11.3 Onboarding a New Building
- 12. System Capabilities
- 13. API Reference
- 14. Technology Stack
- 15. Project Structure
- 16. Future Work

1. Executive Summary

OntoSage is a research-grade **Agentic AI Framework** for smart buildings that enables **Zero-Knowledge Human–Building Interaction (HBI)**. It allows users with no technical expertise — occupants, facility managers, students, and researchers — to interact with complex building systems (sensors, HVAC, lighting) using natural language alone, without needing to understand SPARQL, SQL, ontologies, or IoT protocols.

The system is built on a **multi-agent orchestration** pattern using **LangGraph**, where a central orchestrator decomposes user queries and delegates sub-tasks to specialised agents: a Dialogue Agent for intent detection, a SPARQL Agent for ontology queries, a SQL Agent for time-series data retrieval, an Analytics Agent for statistical computation, and a Visualization Agent for chart generation. All agents coordinate through a unified state machine with conditional routing.

The framework follows an **"Easy-Deploy"** philosophy — containerised via Docker Compose, it can be deployed in any smart building environment with minimal configuration changes. It adapts to the building's existing ontology (Brick Schema, RealEstateCore, ASHRAE 223P, etc.) and database rather than forcing migration.

Key metrics of the current deployment:

Metric	Value
Ontology triples (GraphDB)	365,944
Unique sensors registered	680
Sensor types	20 (temperature, humidity, CO ₂ , PM2.5, etc.)
Time-series records (MySQL)	576,000+
Docker services	18
Supported LLM providers	3 (Ollama local, Ollama Cloud, OpenAI)
API endpoints	20+
Agent types	5 (Dialogue, SPARQL, SQL, Analytics, Visualization)
Intent types	5 (general, metadata, analytics, clarification, discovery)

2. Problem Statement & Motivation

Smart buildings generate vast amounts of data from hundreds of IoT sensors — temperature, humidity, CO₂, occupancy, illuminance, air quality, and more. This data is typically stored across heterogeneous systems:

- **Ontologies** (RDF/OWL) describe the building's physical topology: rooms, zones, floors, HVAC equipment, and their semantic relationships.
- **Relational databases** (MySQL, PostgreSQL) store high-frequency time-series telemetry data from each sensor.
- **IoT platforms** (ThingsBoard, etc.) manage device provisioning and data ingestion.

Accessing this data currently requires expertise in:

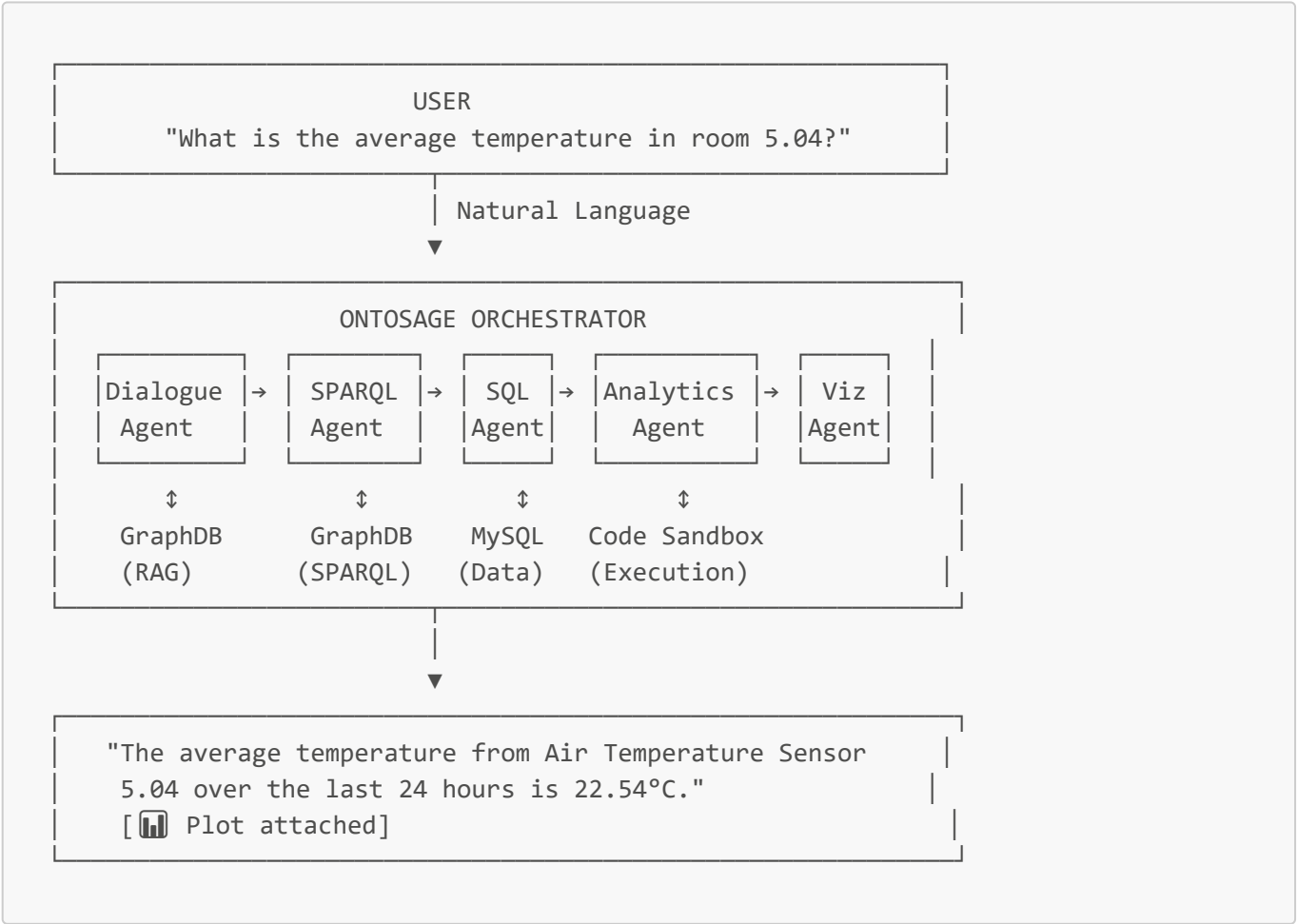
- **SPARQL** for querying the knowledge graph
- **SQL** for retrieving sensor readings
- **Ontology vocabularies** (Brick Schema, ASHRAE 223P) for understanding entity relationships
- **UUID-based identifiers** for mapping between systems

This creates a significant **knowledge barrier**. A facility manager who simply wants to know "What was the average temperature in Room 5.04 yesterday?" must translate that into the correct SPARQL query to find the sensor URI, map it to a UUID, construct the right SQL query against the right database, and interpret raw results.

OntoSage eliminates this barrier entirely. Users ask questions in plain English, and the system handles the full technical translation pipeline autonomously.

3. System Overview

OntoSage operates as a **conversational AI assistant** for buildings. At a high level:



The system supports:

- **General questions** (answered directly by the LLM)
- **Metadata queries** ("What sensors are in zone 5?" — resolved via SPARQL against the ontology)
- **Analytics queries** ("Plot the temperature trend for last week" — resolved via SPARQL → SQL → Python code generation → chart rendering)
- **Clarification requests** (the system asks for more detail when a query is ambiguous)
- **Sensor discovery** (users can explore what sensors and data are available)

4. Architecture

4.1 High-Level Architecture

OntoSage employs a **Hub-and-Spoke agentic architecture**. The Orchestrator is the central cognitive hub that decomposes user queries and delegates to specialised agents.

```
graph TD
    User((User)) -->|Natural Language| Frontend[Frontend  
React 19 + Nginx]
    User -->|Natural Language| OpenWebUI[Open WebUI]

    Frontend -->|REST API / SSE / WebSocket| Orchestrator
    OpenWebUI -->|OpenAI-Compatible API| Orchestrator

    subgraph Orchestrator["Orchestrator Service (FastAPI + LangGraph)"]
        direction TB
        AuthMgr[Auth Manager  
Session + Password]
        Workflow[LangGraph Workflow  
State Machine]
        LLMMgr[LLM Manager  
Provider Abstraction]
        CtxMgr[Context Manager  
Summarization + Pruning]

        AuthMgr --> Workflow
        Workflow --> LLMMgr
        Workflow --> CtxMgr
    end

    subgraph Agents["Multi-Agent System"]
        DA[Dialogue Agent  
Intent Detection]
        SPA[SPARQL Agent  
Ontology Queries]
        SQLA[SQL Agent  
Time-Series Data]
        AA[Analytics Agent  
Code Generation]
        VA[Visualization Agent  
Chart Generation]
    end
```

```
end

    subgraph Knowledge["Knowledge & Data Layer"]
        GraphDB[(GraphDB
365K triples)]
        MySQL[(MySQL
Sensor Data)]
        Redis[(Redis
State + Cache)]
        Postgres[(PostgreSQL
User Data)]
    end

    subgraph Services["Support Services"]
        RAG[RAG Service
GraphDB Similarity]
        CodeExec[Code Executor
Sandboxed Python]
        DataPub[Data Publisher
Dummy Sensor Data]
        FileSrv[File Server
Artifacts / Plots]
    end

    Workflow --> DA
    Workflow --> SPA
    Workflow --> SQLA
    Workflow --> AA
    Workflow --> VA

    DA --> RAG
    SPA --> GraphDB
    SPA --> RAG
    SQLA --> MySQL
    AA --> CodeExec
    VA --> CodeExec

    Orchestrator --> Redis
    Orchestrator --> Postgres
    DataPub --> MySQL
```

4.2 Service Topology

All services run as Docker containers managed by `docker-compose.agentic.yml`. The following table lists every service:

#	Service	Container Name	Port (Host:Container)	Purpose
1	orchestrator	ontosage-orchestrator	8000:8000	Central AI orchestration (FastAPI + LangGraph)

#	Service	Container Name	Port (Host:Container)	Purpose
2	graphdb	graphdb	7200:7200	RDF Knowledge Graph (Ontotext GraphDB 10.7.4)
3	mysql	mysql-bldg1	3307:3306	Time-series sensor data (MySQL 8)
4	redis	redis-memory-store	6379:6379	State cache, sessions, SPARQL result cache
5	postgres-user-data	postgres-user-data	5433:5432	User accounts, conversation history (PostgreSQL 15)
6	graphdb-rag-service	rag-service-graphdb	8001:8001	RAG retrieval via GraphDB Similarity Indexing
7	code-executor	code-executor	8002:8002	Sandboxed Python code execution
8	qdrant	qdrant-vector-db	6333:6333, 6334:6334	Vector database for conversation memory
9	ollama	ollama-deepseek-r1	11435:11434	Local LLM inference (GPU-accelerated)
10	ollama-health	ollama-health	8005:8005	Ollama health check sidecar
11	open-webui	open-webui	3001:8080	Alternative chat UI with voice support
12	data-publisher	data-publisher	—	Continuous dummy sensor data publisher
13	frontend	ontosage-frontend	3000:3000	React 19 web application
14	api	abacws-api	5000:5000	Abacws building management backend
15	visualiser	abacws-visualiser	8090:80	3D building visualisation
16	mongo	mongo-chat-history	27017:27017	Legacy chat history (MongoDB 7)
17	pgadmin	pgadmin	5050:80	PostgreSQL administration UI
18	file-server	file-server-artifacts	8080:80	Static file server for generated plots
19	prometheus	prometheus-metrics	9090:9090	Metrics collection (optional)
20	grafana	grafana-dashboards	3002:3000	Monitoring dashboards (optional)

4.3 Network Architecture

```
graph LR
    subgraph "ontobot-agentic (Bridge Network)"
        Orch[Orchestrator :8000]
        GDB[GraphDB :7200]
        MySQL[MySQL :3306]
        Redis[Redis :6379]
        PG[PostgreSQL :5432]
        RAG[RAG Service :8001]
        CE[Code Executor :8002]
        Qdrant[Qdrant :6333]
        Ollama[Ollama :11434]
        FE[Frontend :3000]
        DP[Data Publisher]
    end

    subgraph "ontobot-network (External)"
        ExtMySQL[MySQL External]
        ExtAPI[Abacws API]
        ExtViz[Abacws Visualiser]
    end

    MySQL --- ExtMySQL
```

Two Docker networks are used:

- **ontobot-agentic** (bridge): Internal communication between all OntoSage services.
- **ontobot-network** (external): Connects MySQL, API, and Visualiser services to external/legacy infrastructure.

5. Core Components

5.1 Orchestrator Service

Location: `orchestrator/`

Technology: Python 3.11, FastAPI 0.115, LangGraph 0.2

Container: `ontosage-orchestrator` (port 8000)

The Orchestrator is the cognitive core of OntoSage. It manages:

1. **API Gateway** — Exposes REST, SSE streaming, WebSocket, and OpenAI-compatible endpoints
2. **Authentication** — Session-based auth with SHA-256 password hashing and Redis session tokens
3. **Workflow Orchestration** — LangGraph state machine controlling agent execution
4. **State Management** — Persists conversation state in Redis and PostgreSQL
5. **LLM Provider Abstraction** — Unified interface for Ollama, OpenAI, and cloud providers

Key Modules:

Module	Purpose
<code>main.py</code> (1,243 lines)	FastAPI application, 20+ API endpoints, lifecycle management
<code>workflow.py</code> (787 lines)	LangGraph <code>StateGraph</code> , agent routing, sensor discovery
<code>llm_manager.py</code> (249 lines)	Provider-agnostic LLM calls with rate limiting
<code>auth_manager.py</code> (519 lines)	User registration, login, session management
<code>redis_manager.py</code> (419 lines)	Conversation state, caching, message history
<code>postgres_manager.py</code> (217 lines)	Persistent user data, chat history (3 tables)
<code>services/context_manager.py</code> (76 lines)	Conversation summarisation, title generation

Conversation State Model (`ConversationState`): A comprehensive Pydantic model that flows through the entire LangGraph pipeline. Key fields include:

```
conversation_id, user_id, title, summary, building_id, persona,
user_message, messages[], current_intent, intent,
needs_clarification, clarification_question,
intermediate_results{}, query_results{},
analytics_required, errors[], retry_count
```

5.2 Multi-Agent System

OntoSage uses five specialised agents coordinated by the LangGraph workflow:

5.2.1 Dialogue Agent

File: `orchestrator/agents/dialogue_agent.py` (452 lines)

The entry point for every user query. Responsibilities:

- **Intent Detection:** Uses the LLM to classify user intent into one of five categories: `general`, `metadata`, `analytics`, `clarification`, or `discovery`
- **Entity Extraction:** Identifies building entities mentioned in the query (e.g., "Air_Temperature_Sensor_5.04", "Zone 5.12")
- **Time Range Parsing:** Extracts temporal references ("last week", "yesterday", "since January")
- **Ontology Context Retrieval:** Calls the RAG Service to fetch relevant ontology fragments for better classification
- **Conversation Context:** Maintains up to 5 recent messages plus a rolling summary for long conversations
- **Persona Adaptation:** Supports four personas (student, researcher, facility_manager, general) with tailored response styles

Intent Classification Prompt: The agent sends a structured prompt to the LLM with:

- Current date/time (UK timezone)
- RAG-retrieved ontology context
- Conversation history (summary + recent messages)
- The user's query

The LLM returns a JSON response with: `intent`, `entities[]`, `required_analytics[]`, `time_range{start, end}`, `response`, `clarification_question`, `discovery_filter`, and `explanation`.

5.2.2 SPARQL Agent

File: `orchestrator/agents/sparql_agent.py` (1,288 lines)

Handles ontology queries against GraphDB. Responsibilities:

- **RAG-Enhanced SPARQL Generation:** Retrieves relevant ontology context via the RAG Service, then uses the LLM to generate syntactically correct SPARQL queries
- **Query Execution:** Sends SPARQL queries to GraphDB's HTTP endpoint
- **Semantic Fallback:** If SPARQL generation fails, falls back to direct LLM reasoning over the retrieved ontology fragments
- **UUID Extraction:** For analytics queries, extracts sensor UUIDs from SPARQL results to pass to the SQL Agent
- **Retry Logic:** Up to 3 retries with error feedback to the LLM for self-correction
- **Result Caching:** SHA-256 hash-based caching in Redis (1-hour TTL)

Supported Ontology Vocabularies (27 prefixes): Brick Schema, ASHRAE 223P, RealEstateCore, SOSA/SSN, Dublin Core, QUDT, SKOS, OWL, RDF/RDFS, BACnet, and the building-specific namespace (`bldg:`).

5.2.3 SQL Agent

File: `orchestrator/agents/sql_agent.py` (578 lines)

Retrieves time-series sensor data from MySQL. Two operational modes:

1. **UUID-Based Fetching** (primary): Receives sensor UUIDs from the SPARQL Agent, validates them against actual database columns, constructs optimised SQL queries with UNION ALL (wide-format to long-format pivoting), and applies time filtering.
2. **Text-to-SQL** (fallback): When no UUIDs are available, uses the LLM to generate SQL directly from the user's natural language query and the database schema.

Key Features:

- **Column Validation:** Validates UUIDs against actual database columns before querying (prevents "Unknown column" SQL errors)
- **Time Reference Parsing:** Handles relative times ("last 24 hours", "this week", "yesterday") and absolute dates
- **Auto-Expand Time Window:** If a query returns zero rows, automatically expands the time window and retries
- **Storage-Aware Routing:** Groups UUIDs by storage location (enables future multi-database support)

- **SQL Injection Prevention:** Only allows SELECT queries; rejects anything containing INSERT, UPDATE, DELETE, DROP, ALTER

5.2.4 Analytics Agent

File: `orchestrator/agents/analytics_agent.py` (627 lines)

Generates and executes Python code for statistical analysis. workflow:

1. **Template Matching:** Checks if the query matches a pre-defined analytics template (average, min/max, trend, count, comparison). Templates avoid unnecessary LLM calls and produce deterministic, tested code.
2. **LLM Code Generation:** For complex or novel queries, uses the LLM to generate custom Python code.
3. **Code Execution:** Sends generated code to the Code Executor service for sandboxed execution.
4. **Result Interpretation:** Uses the LLM to format execution output (statistics, tables) into natural language with sensor label substitution (UUIDs → human-readable names).

Data Pipeline: The agent saves data to a per-user JSON file

(`outputs/data/{user_id}_{conv_id}_data.json`) that the code executor reads — ensuring data isolation between concurrent users.

5.2.5 Visualization Agent

File: `orchestrator/agents/visualization_agent.py` (354 lines)

Generates charts and plots using Matplotlib/Seaborn. Workflow:

1. **Chart Type Detection:** Pattern-based with LLM fallback. Supports: `line_chart`, `bar_chart`, `scatter_plot`, `histogram`, `heatmap`, `pie_chart`.
2. **Code Generation:** LLM generates Matplotlib/Seaborn code tailored to the data and chart type.
3. **Execution:** Code runs in the sandboxed Code Executor; the plot is saved as a PNG.
4. **Embedding:** The generated plot is base64-encoded and embedded directly in the response (no external URL dependency).

5.3 RAG Service (GraphDB Similarity Indexing)

Location: `rag-service/graphdbRAG/`

Technology: FastAPI, httpx, rdflib

Container: `rag-service-graphdb` (port 8001)

Implements the **Ontotext 2-Step Retrieval Pattern** against GraphDB's built-in Similarity Indexing:

```
graph LR
    Query["User Query"] --> Step1["Step 1: Entity Retrieval  
(Vector Similarity Search)"]
    Step1 --> Step2["Step 2: Bounded Context"]
```

```
(1-2 hop SPARQL)"]
    Step2 --> Output["Prefixes + Triples +
Entity Labels + Summary"]

    Step1 -- Index["GraphDB
Similarity Index"]
    Step2 -- Repo["GraphDB
Repository"]
```

Step 1 — Entity Retrieval:

Uses GraphDB's similarity index (`similarity:bldg_index`) to find ontology entities semantically similar to the user's query. Also performs identifier-based retrieval for queries containing specific sensor IDs (e.g., "5.11").

Step 2 — Bounded Context:

For each retrieved entity, executes a SPARQL query to fetch its 1-hop or 2-hop neighbourhood (all triples where the entity appears as subject or object). This provides the LLM with rich structural context.

Auto-Import: On startup, the RAG Service automatically imports `.ttl` files from the `/app/input` directory into GraphDB.

5.4 Code Executor Service (Sandbox)

Location: `code-executor/`
Technology: Python 3.11, FastAPI
Container: `code-executor` (port 8002)

A security-hardened Python execution environment for running analytics and visualisation code. Multiple layers of protection:

Layer	Mechanism
Static Analysis	Regex-based validation rejects forbidden imports and builtins before execution
Import Whitelist	Only safe libraries allowed: <code>pandas</code> , <code>numpy</code> , <code>matplotlib</code> , <code>seaborn</code> , <code>plotly</code> , <code>json</code> , <code>math</code> , <code>statistics</code> , <code>collections</code> , <code>itertools</code> , <code>datetime</code>
Import Blacklist	Blocked: <code>os</code> , <code>sys</code> , <code>subprocess</code> , <code>socket</code> , <code>requests</code> , <code>urllib</code> , <code>pickle</code> , <code>eval</code> , <code>exec</code> , <code>compile</code> , <code>open</code>
Custom Builtins	Replaces Python builtins with a safe whitelist (30+ items)
Execution Isolation	Runs in <code>ThreadPoolExecutor</code> with <code>asyncio.wait_for()</code> timeout
I/O Capture	<code>stdout/stderr</code> redirected to <code>StringIO</code> buffers
Resource Limits	Docker: 1 CPU, 1 GB memory, <code>no-new-privileges</code> security option

Layer	Mechanism
No Shell Access	Container user has <code>/usr/sbin/nologin</code> as shell

Endpoints:

- `POST /execute` — Execute Python code with timeout and context
- `POST /validate` — Syntax-only validation via `ast.parse()`
- `GET /health` — Health check

5.5 Frontend Application

Location: `frontend/`
Technology: React 19, Bootstrap 5, Axios, Nginx
Container: `ontosage-frontend` (port 3000)

Features:

- **Chat Interface:** Streaming responses (SSE), markdown rendering, code highlighting
- **Media Rendering:** Displays inline plots and charts from base64-encoded images
- **Conversation History:** Sidebar with conversation list, create/select/delete
- **Voice Input:** MediaRecorder API → Whisper STT → text processing
- **Authentication:** Login/register forms, session token management
- **Offline Storage:** Dexie (IndexedDB) fallback for offline scenarios
- **Health Dashboard:** System health monitoring page

6. Data Layer

6.1 GraphDB (Knowledge Graph)

Image: Ontotext GraphDB 10.7.4
Repository: `bldg`
Triples: 365,944
Ontology Standard: Brick Schema + ASHRAE 223P

The GraphDB instance stores the building's **semantic model** as an RDF knowledge graph. It describes:

- **Physical Topology:** Site → Building → Floor → Zone → Room
- **Equipment:** HVAC systems, lighting, metering
- **Sensors:** 680 unique sensors across 20 types
- **Relationships:** `brick:hasPoint`, `brick:isPartOf`, `brick:feeds`, `brick:isLocationOf`
- **External References:** `ashrae:hasExternalReference` linking sensors to time-series UUIDs and storage locations
- **Labels:** Human-readable `rdfs:label` for each entity

Similarity Index: A GraphDB-native vector index (`bldg_index`) enables semantic search directly within the knowledge graph — no external vector database required for ontology lookups.

Supported Query Operations:

```
# Example: Find all temperature sensors in zone 5
PREFIX brick: <https://brickschema.org/schema/Brick#>
PREFIX bldg: <http://abacwsbuilding.cardiff.ac.uk/abacws#>

SELECT ?sensor ?label ?uuid WHERE {
  ?sensor rdf:type brick:Air_Temperature_Sensor .
  ?sensor rdfs:label ?label .
  ?sensor ashrae:hasExternalReference ?ref .
  ?ref ref:hasTimeseriesId ?uuid .
}
```

6.2 MySQL (Time-Series Sensor Data)

Image: MySQL 8
Database: `sensordb`
Table: `sensor_data`
Records: 576,000+

Stores high-frequency sensor telemetry in a **wide-format** table where each column is a sensor UUID:

Column	Type
<code>Datetime</code>	TIMESTAMP (primary time index)
<code><uuid-1></code>	FLOAT / VARCHAR (sensor value)
<code><uuid-2></code>	FLOAT / VARCHAR (sensor value)
...	... (680 UUID columns)

The SQL Agent unpivots this wide format into a standard long format (`uuid`, `value`, `timestamp`) using UNION ALL queries for analytics processing.

Data Publisher Service (`data-publisher`): A dedicated Docker service that continuously inserts realistic dummy sensor data every 30 seconds, simulating live IoT data ingestion. It reads sensor UUIDs from `sensor_uuids.json` and generates physically plausible values (e.g., temperature: 18–26°C, humidity: 30–70%, CO₂: 400–1200 ppm).

6.3 Redis (State & Cache)

Image: Redis 7 Alpine
Configuration: `appendonly` `yes`, 2 GB max memory, LRU eviction

Redis serves as the system's **fast memory layer**:

Key Pattern	Purpose	TTL
<code>conversation:{id}</code>	Serialised ConversationState JSON	1 hour
<code>messages:{conversation_id}</code>	Message history list	1 hour

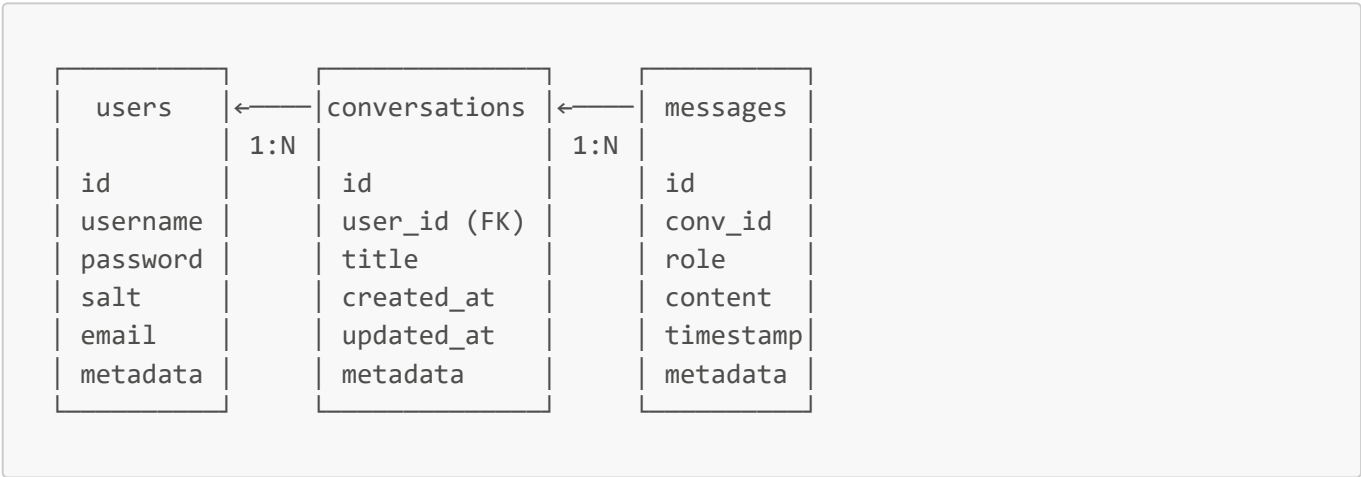
Key Pattern	Purpose	TTL
<code>session:{token}</code>	User session data	7 days
<code>user_sessions:{username}</code>	Set of active session tokens	7 days
<code>user:{username}</code>	User profile data	—
<code>cache:sparql:{hash}</code>	Cached SPARQL results	1 hour
<code>cache:intent:{hash}</code>	Cached intent detection results	1 hour
<code>user:prefs:{user_id}</code>	User preferences JSON	30 days
<code>user:{id}:conversations</code>	Set of user's conversation IDs	—

6.4 PostgreSQL (User Data & History)

Image: PostgreSQL 15 Alpine

Database: `ontobot`

Schema: 3 tables, auto-created on startup



6.5 Sensor Map Cache

File: `data/sensor_map.json` (12,242 lines, 2,040 entries)

A pre-computed cache that maps sensor identifiers (by name, label, and URI) to their metadata:

```
{
  "Air_Temperature_Sensor_5.04": {
    "uri":
      "http://abacwsbuilding.cardiff.ac.uk/abacws#Air_Temperature_Sensor_5.04",
    "uuid": "a1b2c3d4-...",
    "storage": "http://abacwsbuilding.cardiff.ac.uk/abacws#database1",
    "label": "Air Temperature Sensor installed-node 5.04"
  }
}
```

Generated by `scripts/cache_sensor_map.py` which queries GraphDB for all sensors with `ashrae:hasExternalReference`. Used by the workflow for:

- **Sensor Discovery:** Listing available sensors by type when users ask "what sensors do you have?"
- **UUID-to-Name Translation:** Replacing UUIDs with human-readable names in analytics responses

7. Workflow: End-to-End Query Resolution

7.1 LangGraph State Machine

The workflow is implemented as a **LangGraph StateGraph** — a directed graph where nodes are agent functions and edges are conditional routing decisions.

```
stateDiagram-v2
    [*] --> Dialogue

    Dialogue --> Response : general / clarification / discovery
    Dialogue --> SPARQL : metadata
    Dialogue --> SPARQL : analytics (needs UUIDs)
    Dialogue --> SQL : sql (direct)
    Dialogue --> Visualization : visualization (direct)

    SPARQL --> Response : metadata only
    SPARQL --> SQL : analytics_required = true
    SPARQL --> Visualization : viz keywords detected

    SQL --> Analytics : analytics_required = true
    SQL --> Response : data only
    SQL --> Visualization : viz keywords detected

    Analytics --> Response : PLOT_GENERATED in output
    Analytics --> Visualization : viz keywords detected
    Analytics --> Response : analysis complete

    Visualization --> Response

    Response --> [*]
```

7.2 Intent Classification & Routing

The Dialogue Agent classifies every user query into one of five intents:

Intent	Description	Route
<code>general</code>	General knowledge, greetings, small talk	Dialogue → Response
<code>metadata</code>	Questions about building structure, sensor locations, equipment	Dialogue → SPARQL → Response

Intent	Description	Route
analytics	Questions about sensor values, trends, statistics	Dialogue → SPARQL → SQL → Analytics → Response
clarification	Query is too vague; system needs more information	Dialogue → Response (with a follow-up question)
discovery	User wants to explore available sensors/capabilities	Dialogue → Response (with sensor listing)

7.3 Query Resolution Flows

Flow 1: General Question

```
User: "What is Brick Schema?"
→ Dialogue Agent: intent=general, response="Brick Schema is..."
→ Response Node: returns direct answer
```

Flow 2: Metadata Query

```
User: "List all sensors in zone 5.04"
→ Dialogue Agent: intent=metadata, entities=["Zone 5.04"]
→ SPARQL Agent:
  1. RAG retrieves ontology context about Zone 5.04
  2. LLM generates SPARQL: SELECT ?sensor WHERE { ?sensor brick:isPartOf
bldg:Zone_5.04 }
  3. Executes against GraphDB
  4. Formats results into natural language
→ Response Node: "Zone 5.04 has the following sensors: ..."
```

Flow 3: Analytics Query (Full Pipeline)

```
User: "What is the average temperature in room 5.04?"
→ Dialogue Agent: intent=analytics, entities=["Air_Temperature_Sensor_5.04"],
  required_analytics=["avg"]
→ SPARQL Agent:
  1. Generates SPARQL to find sensor UUID + storage location
  2. Executes: finds UUID "a1b2c3d4-..."
→ SQL Agent:
  1. Validates UUID exists as column in MySQL
  2. Generates time-ranged SQL query (auto-selects recent data)
  3. Returns data: [{uuid, value, timestamp}, ...]
→ Analytics Agent:
  1. Saves data to outputs/data/{user}_{conv}_data.json
  2. Generates Python code:
    ```python
```



```
import pandas as pd
df = pd.read_json('/app/outputs/data/...')
avg = df['value'].mean()
print(f"Mean Value: {avg:.2f}°C")
```


3. Sends code to Code Executor → sandboxed execution
4. LLM formats result: "The average temperature is 22.54°C"
5. If plot generated, base64-encodes and embeds in response



→ Response Node: "The average temperature from Air Temperature Sensor 5.04 is 22.54°C. [ Plot]"


```

Flow 4: Clarification

User: "Show me the temperature"

→ Dialogue Agent: intent=clarification
 clarification_question="Which temperature sensor?
 Did you mean Air_Temperature_Sensor_5.04 or Air_Temperature_Sensor_5.12?"

→ Response Node: returns the clarification question

User: "5.04"

→ (Next turn) Dialogue Agent: intent=analytics (with conversation history context)

→ [Continues with Flow 3]

7.4 Clarification & Sensor Discovery

Clarification is triggered when:

- The user mentions a sensor type but not which specific sensor
- The query could be metadata OR analytics and the system cannot determine which
- Critical details are missing (e.g., "compare sensors" — which sensors? what metric?)

Sensor Discovery enables users to explore what's available:

User: "What sensors do you have?"

→ Discovery (unfiltered): Lists 680 sensors grouped by 20 types

Output: "Found 680 sensors:
 - Air Temperature Sensor: 33 sensors
 - CO2 Level Sensor: 33 sensors
 - Humidity Sensor: 33 sensors
 - PM2.5 Level Sensor: 33 sensors
 ..."

User: "What temperature sensors are available?"

→ Discovery (filtered by "temperature"): Lists 34 matching sensors

Output: "Found 34 sensors matching 'temperature':
 - Air Temperature Sensor: 33 sensors
 - Air Temperature Sensor 5.02: 1 sensor"

8. LLM Integration & Provider Flexibility

OntoSage supports three LLM providers through a unified `LLMManager` abstraction:

| Provider | Model | Use Case | Configuration |
|----------------|-----------------------|--|--|
| Ollama (Local) | DeepSeek-R1:32b | Privacy-focused, free, GPU-accelerated | <code>MODEL_PROVIDER=local</code> |
| OpenAI | GPT-4o-mini / o3-mini | High performance, cloud-based | <code>MODEL_PROVIDER=openai</code> + API key |
| Ollama Cloud | Custom | Scalable, intermediate | <code>MODEL_PROVIDER=cloud</code> + API key |

Provider Switching: Change `MODEL_PROVIDER` in `.env` and restart the orchestrator. No code changes required.

Rate Limiting: For OpenAI, a configurable delay (default 1 second) is enforced between API calls to respect rate limits. Controlled via `OPENAI_RATE_LIMIT_DELAY` environment variable.

LLM Usage Points in the pipeline:

- Intent Detection** (Dialogue Agent) — classify user query
- SPARQL Generation** (SPARQL Agent) — generate ontology queries
- SQL Generation** (SQL Agent) — generate time-series queries
- Code Generation** (Analytics Agent) — generate Python analytics code
- Result Formatting** (Analytics Agent) — format raw results into natural language
- Chart Type Detection** (Visualization Agent) — determine chart type
- Visualisation Code** (Visualization Agent) — generate Matplotlib code
- Context Summarisation** (Context Manager) — summarise long conversations
- Title Generation** (Context Manager) — auto-title new conversations

9. Authentication & Security

Authentication Flow



```
O-->>F: Registration success

U->>F: Login (username, password)
F->>O: POST /auth/login
O->>P: Verify credentials
O->>R: Create session (secrets.token_urlsafe(32))
O-->>F: Session token (7-day TTL) + Set-Cookie

U->>F: Chat message
F->>O: POST /chat (Authorization: Bearer )
O->>R: Validate session token
O->>O: Process through LangGraph workflow
O-->>F: Response
```

Security Measures

| Area | Mechanism |
|--------------------------|--|
| Password Storage | SHA-256 with random 16-byte hex salt per user |
| Session Management | Cryptographically random 32-byte tokens, 7-day TTL, Redis-backed |
| Code Execution | Sandboxed: import whitelist, builtin restrictions, resource limits, no shell |
| SQL Injection Prevention | SELECT-only validation, parameterised queries |
| CORS | Configurable (needs restriction for production) |
| Container Security | Non-root users, no-new-privileges , memory/CPU limits |
| Network Isolation | Internal bridge network; services not exposed on host except API ports |

10. Monitoring & Observability

Prometheus + Grafana (Optional)

Enabled via the monitoring Docker Compose profile:

| Target | Endpoint | Interval |
|---------------|----------------------------|----------|
| Orchestrator | :8000/metrics | 10s |
| RAG Service | :8001/metrics | 10s |
| Code Executor | :8002/metrics | 10s |
| Redis | :6379 | 15s |
| Qdrant | :6333/metrics | 15s |
| MySQL | :3306 | 15s |

Structured Logging

All services use Python's `logging` module with a consistent format:

```
2026-02-18 22:05:20,408 - orchestrator.workflow - INFO - Loaded 2040 sensors from cache
```

The Orchestrator emits detailed progress logs with emoji markers:

- 📄 User query received
- 🔍 RAG context retrieval
- 🤖 LLM intent detection
- 📊 Intent analysis results
- 🔄 Clarification needed
- 💾 SQL data fetching
- 📈 Analytics processing
- 📝 Response formatting
- ✅ / ❌ Success / failure markers

Health Endpoints

Every service exposes a `GET /health` endpoint:

| Service | Health Check |
|---------------|---|
| Orchestrator | Redis connectivity + model provider info |
| RAG Service | GraphDB connection + repository status |
| Code Executor | Sandbox configuration (timeout, memory, CPU limits) |
| GraphDB | Repository listing API |
| MySQL | <code>mysqladmin ping</code> |
| Redis | <code>redis-cli ping</code> |
| PostgreSQL | <code>pg_isready</code> |

Aggregate health: `GET /health/aggregate` on the Orchestrator checks all downstream services.

11. Deployment

11.1 Prerequisites & Hardware Requirements

Minimum Requirements (Cloud LLM — OpenAI):

| Resource | Requirement |
|----------|---|
| OS | Windows 10/11, Ubuntu 20.04+, macOS 12+ |
| Docker | Docker Desktop 4.x or Docker Engine 24+ |

| Resource | Requirement |
|----------|--|
| RAM | 16 GB |
| Disk | 20 GB (for Docker images + data volumes) |
| Network | Internet access for OpenAI API |
| API Key | OpenAI API key with GPT-4 access |

Recommended Requirements (Local LLM — Ollama DeepSeek-R1:32b):

| Resource | Requirement |
|----------------|---|
| OS | Windows 10/11, Ubuntu 20.04+, macOS 12+ |
| Docker | Docker Desktop 4.x with GPU support |
| GPU | NVIDIA GPU with $\geq$ 16 GB VRAM (RTX 3090/4090) |
| RAM | 64 GB |
| Disk | 50 GB (includes ~20 GB model download) |
| NVIDIA Drivers | nvidia-docker2 / NVIDIA Container Toolkit installed |

Software Prerequisites:

- Docker Desktop or Docker Engine + Docker Compose v2
- Git
- (Optional) PowerShell 7+ or Bash for helper scripts
- (Optional) NVIDIA Container Toolkit for GPU support

Data Prerequisites:

- A building ontology file in **Turtle (.ttl) format** describing the building's physical structure, sensor types, and their relationships using a standard vocabulary (Brick Schema recommended)
- A time-series database (MySQL) with sensor telemetry data, where column names are sensor UUIDs
- (Optional) A `sensor_uuids.json` mapping sensor names to UUIDs for the data publisher

11.2 Step-by-Step Setup

1. Clone the Repository

```
git clone https://github.com/suhasdevmane/OntoBot.git
cd OntoBot
```

2. Configure Environment

```
# For OpenAI (recommended for first-time setup):
cp .env.example .env
```

```
# Edit .env and set:
#   MODEL_PROVIDER=openai
#   OPENAI_API_KEY=sk-your-key-here

# For local Ollama:
# Edit .env and set:
#   MODEL_PROVIDER=local
```

3. Create Volume Directories

```
mkdir -p volumes/ollama volumes/qdrant volumes/redis volumes/graphdb
mkdir -p volumes/mongo volumes/artifacts volumes/postgres-user-data
```

4. Prepare the Ontology

Place your building's `.ttl` file in the `input/` directory:

```
cp /path/to/your/building_ontology.ttl input/
```

The RAG Service will automatically import it into GraphDB on startup.

5. Create the External Docker Network and Volume

```
docker network create ontobot-network
docker volume create --name ontobot_mysql-data
```

6. Start All Services

```
docker-compose -f docker-compose.agentic.yml up -d --build
```

This starts all 18+ services. The first build takes 5–10 minutes.

7. (Local Only) Pull the LLM Model

```
docker exec ollama-deepseek-r1 ollama pull deepseek-r1:32b
# This downloads ~20 GB. Takes 10–15 minutes.
```

8. Generate the Sensor Map Cache

```
# Wait for GraphDB to fully import the ontology (~2 minutes), then:
python scripts/cache_sensor_map.py
```

9. Create a Health Check Placeholder

```
echo "<html><body>OK</body></html>" > volumes/artifacts/index.html
```

10. Verify Health

```
# From PowerShell:
pwsh scripts/check-health.ps1

# Or manually:
curl http://localhost:8000/health
curl http://localhost:8001/health
curl http://localhost:8002/health
curl http://localhost:7200/rest/repositories
```

11. Access the System

| Service | URL |
|------------|----------------------------|
| Frontend | http://localhost:3000 |
| API Docs | http://localhost:8000/docs |
| GraphDB UI | http://localhost:7200 |
| pgAdmin | http://localhost:5050 |
| Prometheus | http://localhost:9090 |
| Grafana | http://localhost:3002 |

11.3 Onboarding a New Building

OntoSage is designed to be **building-agnostic**. To deploy for a different building:

Step 1: Prepare the Ontology (.ttl)

Your ontology file must define at minimum:

```
@prefix brick: <https://brickschema.org/schema/Brick#> .
@prefix bldg: <http://your-building-namespace#> .
@prefix rdfs: <http://www.w3.org/2000/01/rdf-schema#> .
@prefix ashrae: <http://data.ashrae.org/standard223#> .
```

```
@prefix ref: <https://brickschema.org/schema/Brick/ref#> .

# Physical Structure
bldg:MyBuilding a brick:Building .
bldg:Floor1 a brick:Floor ; brick:isPartOf bldg:MyBuilding .
bldg:Room101 a brick:Room ; brick:isPartOf bldg:Floor1 .

# Sensors with UUID linkage
bldg:TempSensor1 a brick:Air_Temperature_Sensor ;
  rdfs:label "Temperature Sensor Room 101" ;
  brick:isPointOf bldg:Room101 ;
  ashrae:hasExternalReference [
    ref:hasTimeseriesId "uuid-from-your-database" ;
    ref:storedAt bldg:database1
  ] .
```

Critical requirement: Each sensor that has time-series data in the database **must** have an `ashrae:hasExternalReference` linking its `ref:hasTimeseriesId` (UUID) and `ref:storedAt` (storage location).

Step 2: Prepare the Database

Your MySQL database should have a table where each column represents a sensor UUID and rows contain timestamped readings:

| Datetime | uuid-1 | uuid-2 | ... |
|---------------------|--------|--------|-----|
| 2026-01-01 00:00:00 | 22.5 | 45.2 | ... |

Step 3: Update Configuration

- 1. Place your `.ttl` file in `input/`
- 2. Update `bldg:` prefix in the SPARQL Agent if your namespace differs
- 3. Update MySQL credentials in `.env` if different
- 4. Regenerate the sensor map: `python scripts/cache_sensor_map.py`

Step 4: Restart Services

```
docker-compose -f docker-compose.agentic.yml down
docker-compose -f docker-compose.agentic.yml up -d --build
```

12. System Capabilities

What Users Can Do

| Capability | Example Query | Agents Involved |
|------------|---------------|-----------------|
|------------|---------------|-----------------|

| Capability | Example Query | Agents Involved |
|------------------------------|---|---|
| Greeting & General Knowledge | "Hello", "What is HVAC?" | Dialogue |
| Sensor Discovery | "What sensors do you have?" | Dialogue (sensor map) |
| Filtered Discovery | "List all temperature sensors" | Dialogue (filtered sensor map) |
| Building Topology | "What rooms are on floor 5?" | Dialogue → SPARQL |
| Sensor Location | "Where is Air_Temperature_Sensor_5.04?" | Dialogue → SPARQL |
| Equipment Details | "What type of sensor is in room 5.12?" | Dialogue → SPARQL |
| Current Readings | "What is the current temperature in 5.04?" | Dialogue → SPARQL → SQL → Analytics |
| Historical Data | "Show temperature data from last week" | Dialogue → SPARQL → SQL → Analytics |
| Statistical Analysis | "Average humidity in zone 5 for January" | Dialogue → SPARQL → SQL → Analytics |
| Trend Analysis | "Plot the temperature trend for last month" | Dialogue → SPARQL → SQL → Analytics → Viz |
| Comparison | "Compare CO ₂ levels between room 5.04 and 5.12" | Dialogue → SPARQL → SQL → Analytics → Viz |
| Correlation | "Correlation between temperature and humidity" | Dialogue → SPARQL → SQL → Analytics → Viz |
| Clarification | "Show me the temperature" (vague) | Dialogue → Response (asks which sensor) |
| Follow-up Questions | "What about humidity?" (in context) | Conversation history resolves context |
| Voice Input | Spoken query via microphone | Whisper STT → same pipeline |

Supported Sensor Types (Current Deployment)

| Sensor Type | Count | Data Type |
|-----------------------|-------|----------------------------|
| Air Temperature | 33 | Float (°C) |
| Zone Air Humidity | 33 | Float (%) |
| CO ₂ Level | 33 | Float (ppm) |
| CO Level | 33 | Float (ppm) |
| PM2.5 Level | 33 | Float (µg/m ³) |

| Sensor Type | Count | Data Type |
|--|-------|---------------------------------|
| PM10 Level | 33 | Float (µg/m³) |
| PM1 Level | 33 | Float (µg/m³) |
| NO ₂ Level | 33 | Float (ppb) |
| Formaldehyde Level | 33 | Float (ppb) |
| TVOC Level | 33 | Float (ppb) |
| Oxygen | 33 | Float (%) |
| Illuminance | 33 | Float (lux) |
| Sound/Loudness | 33 | Float (dB) |
| Air Quality | 33 | Integer (AQI) |
| Air Quality Level | 33 | Enum (low/medium/high/freshair) |
| MQ2 Gas | 33 | Float |
| MQ3 Gas | 33 | Float |
| MQ5 Gas | 33 | Float |
| MQ9 Gas | 33 | Float |
| Ethyl Alcohol/C ₂ H ₅ OH | 33 | Float |

13. API Reference

Primary Endpoints

| Endpoint | Method | Auth | Description |
|----------------------|-----------|---------|---|
| /chat | POST | Yes | Main synchronous chat (LangGraph pipeline) |
| /chat/stream | POST | Yes | Server-Sent Events streaming chat |
| /stream | WebSocket | Yes | Real-time WebSocket streaming with progress |
| /v1/chat/completions | POST | API Key | OpenAI-compatible endpoint (for Open WebUI) |
| /v1/models | GET | — | Lists available models (OpenAI-compatible) |

Authentication Endpoints

| Endpoint | Method | Auth | Description |
|----------------|--------|------|------------------------------|
| /auth/register | POST | No | User registration |
| /auth/login | POST | No | Login, returns session token |
| /auth/logout | POST | Yes | Invalidate session |

| Endpoint | Method | Auth | Description |
|----------|--------|------|-------------------|
| /auth/me | GET | Yes | Current user info |

Data Endpoints

| Endpoint | Method | Auth | Description |
|-----------------------------------|-----------------|------|---------------------------------|
| /conversations/{user_id} | GET | Yes | List user's conversations |
| /conversations/{conv_id}/messages | GET | Yes | Get messages for a conversation |
| /conversation/{id} | GET | Yes | Get conversation by ID |
| /conversation/{id} | DELETE | Yes | Delete conversation |
| /history/{username} | GET/POST/DELETE | Yes | Chat history management |
| /preferences | POST | Yes | Update user preferences |

System Endpoints

| Endpoint | Method | Auth | Description |
|-------------------|--------|------|-------------------------------------|
| /health | GET | No | Service health check |
| /health/aggregate | GET | No | Comprehensive health (all services) |
| /static/{file} | GET | No | Serve generated plots/artifacts |

Chat Request Format

```
{
  "message": "What is the average temperature in room 5.04?",
  "conversation_id": "conv_abc123",    // optional, auto-generated if omitted
  "user_id": "testuser",
  "persona": "guest"                  // guest | stakeholder | officer |
  facility_manager
}
```

Chat Response Format

```
{
  "success": true,
  "data": {
    "conversation_id": "conv_abc123",
    "response": "The average temperature from Air Temperature Sensor 5.04 is
22.54°C.\n\n[Plot](data:image/png;base64,...)",
    "intent": "analytics",
    "username": "testuser",
  }
}
```

```
    "analytics": true,
    "media": [{"type": "image", "url": "data:image/png;base64,...", "filename":
"viz_abc123.png"}]
  },
  "error": null,
  "meta": null
}
```

14. Technology Stack

Backend

| Technology | Version | Purpose |
|------------|---------|---|
| Python | 3.11 | Primary language |
| FastAPI | 0.115.5 | REST API framework (Orchestrator, Code Executor, RAG Service) |
| LangGraph | 0.2.55 | Multi-agent state machine orchestration |
| LangChain | 0.3.13 | LLM abstraction (OpenAI, Ollama integration) |
| Pydantic | 2.10.3 | Data validation, settings management |
| Uvicorn | 0.32.1 | ASGI server |
| httpx | 0.28.1 | Async HTTP client |
| aiomysql | 0.2.0 | Async MySQL driver |
| asyncpg | 0.29.0 | Async PostgreSQL driver |
| redis | 5.2.0 | Async Redis client |
| rdflib | 7.0.0 | RDF parsing and SPARQL (RAG Service) |
| PyMySQL | 1.1.1 | MySQL driver (Data Publisher) |

Frontend

| Technology | Version | Purpose |
|--------------|---------|---------------------------------|
| React | 19 | UI framework |
| Bootstrap | 5.3.3 | CSS framework |
| Axios | 1.8.4 | HTTP client |
| React Router | 7.4.0 | Routing |
| Dexie | 4.0.11 | IndexedDB ORM (offline storage) |

Data Analytics (Code Executor)

| Library | Version | Purpose |
|------------|---------|---------------------------|
| Pandas | 2.1.4 | Data manipulation |
| NumPy | 1.26.2 | Numerical computing |
| Matplotlib | 3.8.2 | Chart generation |
| Seaborn | 0.13.0 | Statistical visualisation |
| Plotly | 5.18.0 | Interactive charts |

Infrastructure

| Technology | Version | Purpose |
|------------------|-----------|---|
| Docker | 24+ | Containerisation |
| Docker Compose | v2 | Service orchestration |
| Ontotext GraphDB | 10.7.4 | RDF Knowledge Graph + Similarity Indexing |
| MySQL | 8 | Time-series sensor data |
| PostgreSQL | 15 Alpine | User data persistence |
| Redis | 7 Alpine | State cache, sessions |
| Qdrant | Latest | Vector database (conversation memory) |
| MongoDB | 7 | Legacy chat history |
| Nginx | Alpine | Frontend serving, reverse proxy |
| Ollama | Latest | Local LLM hosting (GPU-accelerated) |
| Prometheus | Latest | Metrics collection |
| Grafana | Latest | Monitoring dashboards |

15. Project Structure

```
OntoBot/
├── orchestrator/                # Core AI orchestration service
│   ├── main.py                 # FastAPI app, 20+ endpoints (1,243 lines)
│   ├── workflow.py            # LangGraph state machine (787 lines)
│   ├── llm_manager.py         # LLM provider abstraction (249 lines)
│   ├── auth_manager.py        # Authentication & sessions (519 lines)
│   ├── redis_manager.py       # State & cache management (419 lines)
│   ├── postgres_manager.py    # User data persistence (217 lines)
│   ├── Dockerfile             # Python 3.11-slim, non-root user
│   ├── requirements.txt       # Python dependencies
│   └── agents/
│       ├── dialogue_agent.py  # Intent detection & persona (452 lines)
│       └── sparql_agent.py    # Ontology queries + RAG (1,288 lines)
```

```

├── sql_agent.py          # Time-series data (578 lines)
├── analytics_agent.py    # Code generation (627 lines)
├── visualization_agent.py # Chart generation (354 lines)
├── __init__.py           # Agent exports
├── services/
│   └── context_manager.py # Summarisation & pruning (76 lines)
├── shared/               # Shared code across services
│   ├── models.py         # Pydantic models (275 lines)
│   ├── config.py         # Centralised configuration (266 lines)
│   └── utils.py          # Utilities: logging, hashing, parsing (210 lines)
├── rag-service/          # RAG retrieval service
│   ├── graphdbRAG/
│   │   ├── main.py       # FastAPI app (166 lines)
│   │   ├── graphdb_retriever.py # 2-step retrieval (546 lines)
│   │   ├── import_ontology.py # Auto-import .ttl files (89 lines)
│   │   ├── Dockerfile
│   │   └── requirements.txt
│   └── graphdb_retriever.py # 2-step retrieval (546 lines)
├── code-executor/        # Sandboxed Python execution
│   ├── main.py           # FastAPI app (124 lines)
│   ├── sandbox.py        # Security sandbox (319 lines)
│   ├── Dockerfile
│   └── requirements.txt
├── frontend/             # React 19 web application
│   ├── src/
│   │   ├── components/   # ChatBot, Login, MediaRenderer, etc.
│   │   ├── pages/        # Settings, Health, Analytics
│   │   ├── api/          # API client
│   │   └── App.js         # Root component
│   ├── Dockerfile        # Node 20 build → Nginx serve
│   └── package.json
├── mysql-dummy-publish-dev/ # Dummy data publisher
│   ├── mysql_dummy_publisher.py # Continuous publisher (471 lines)
│   ├── sensor_uuids.json       # Sensor UUID mapping
│   ├── Dockerfile
│   └── postgresql columns.csv
├── data/                 # Data files
│   ├── sensor_map.json      # Pre-cached sensor metadata (12,242 lines)
│   └── mysql-init/          # MySQL initialisation scripts
├── input/                # Ontology files for auto-import
│   └── *.ttl               # Building ontology (Turtle format)
├── scripts/              # Helper scripts
│   ├── cache_sensor_map.py  # Generate sensor_map.json from GraphDB
│   ├── check-health.ps1     # PowerShell health checker
│   └── check-health.sh      # Bash health checker
├── monitoring/           # Observability

```

```
├── prometheus.yml          # Prometheus configuration
├── grafana-dashboards/     # Grafana dashboard definitions
├── docs/                  # Documentation
│   ├── ARCHITECTURE.md
│   ├── SERVICES.md
│   ├── CONFIGURATION.md
│   ├── BUILDING_ONBOARDING.md
│   ├── DEVELOPER_GUIDE.md
│   ├── USER_GUIDE.md
│   ├── RUNBOOK.md
│   └── SECURITY.md
├── docker-compose.agentic.yml # Main Docker Compose (826 lines)
├── .env                   # Environment configuration
├── SYSTEM_REPORT.md       # This document
└── README.md              # Project README
```

16. Future Work

16.1 Short-Term Improvements

| Area | Improvement | Impact |
|-------------------------|--|--|
| Multi-Building Support | Extend the system to handle multiple buildings simultaneously with dynamic building context switching | Enables campus-wide deployment |
| Streaming Analytics | Real-time data processing with Apache Kafka or MQTT for live sensor updates | Enables alerting and real-time dashboards |
| Agent Memory | Per-agent long-term memory (currently only system-level via Redis/Qdrant) for learning preferred query patterns | Improves response quality over time |
| Ontology Auto-Detection | Automatically detect ontology vocabulary (Brick, RealEstateCore, ASHRAE 223P, etc.) from the <code>.ttl</code> file and adapt prefixes | Truly zero-configuration building onboarding |
| Caching Improvements | Smarter cache invalidation for SPARQL/intent results; cache analytics results for repeated queries | Reduces LLM API costs and latency |
| Password Security | Migrate from SHA-256 to bcrypt/Argon2 for password hashing | Industry-standard security |

16.2 Medium-Term Enhancements

| Area | Enhancement | Impact |
|------|-------------|--------|
|------|-------------|--------|

| Area | Enhancement | Impact |
|------------------------|--|--|
| Multi-Database Support | Extend SQL Agent to query PostgreSQL, TimescaleDB, InfluxDB, and Cassandra based on the <code>ref:storedAt</code> property in the ontology | Adapt to any building's existing database infrastructure |
| Conversational Repair | Allow the system to detect and recover from failed queries by rephrasing SPARQL/SQL automatically, with user-in-the-loop for ambiguous cases | Higher success rate on complex queries |
| Proactive Insights | System-initiated alerts ("Temperature in Room 5.04 has been above 28°C for 2 hours") based on anomaly detection | Shifts from reactive to proactive facility management |
| Federated Learning | Train a specialised small model on successful query-response pairs to reduce LLM dependency over time | Lower cost, faster responses, offline capability |
| Multi-Modal Input | Image input (e.g., photo of a room label → identify room in ontology) in addition to text and voice | More natural interaction |
| RBAC & Multi-Tenancy | Role-based access control beyond personas (admin, read-only, department-scoped) | Enterprise deployment readiness |

16.3 Long-Term Research Directions

| Direction | Description |
|------------------------------------|---|
| Cross-Building Transfer Learning | Train ontology-agnostic agent models that can be deployed in a new building with zero-shot or few-shot adaptation |
| Digital Twin Integration | Connect OntoSage to real-time 3D digital twin platforms (e.g., Unity, Unreal Engine) for spatial-aware interactions |
| Human-Building Interaction Studies | Conduct user studies to measure the impact of zero-knowledge interaction on occupant satisfaction, energy savings, and facility management efficiency |
| Autonomous Control | Extend from read-only querying to actuation (e.g., "lower the temperature by 2 degrees") with safety guardrails and human-in-the-loop approval |
| Multi-Agent Negotiation | Enable agents to negotiate conflicting objectives (e.g., comfort vs. energy efficiency) and present trade-offs to users |
| Benchmark Dataset | Create a standardised benchmark for evaluating HBI systems across different building ontologies, sensor types, and query complexities |
| Edge Deployment | Package OntoSage for edge devices (Raspberry Pi, NVIDIA Jetson) for low-latency local deployment without cloud dependencies |

16.4 Making the System More Robust & Adaptable

To make OntoSage production-ready for diverse environments:

1. **Configuration-as-Code:** Replace hardcoded namespace prefixes (e.g., `bldg:`) with a configuration file that maps the building's namespace. This eliminates the need to edit agent source code when deploying for a new building.
2. **Schema Discovery:** Instead of a fixed MySQL table structure, the SQL Agent should dynamically discover the database schema (table names, column types) and adapt queries accordingly.
3. **Ontology Validation:** Add a startup validation step that checks the loaded ontology for required patterns (sensors with `hasExternalReference`, labels, etc.) and reports missing elements.
4. **Automated Testing:** Implement integration tests that spin up a minimal Docker environment, load a test ontology, and verify end-to-end query resolution for all five intent types.
5. **Graceful Degradation:** When a downstream service is unavailable (e.g., GraphDB is down), the system should gracefully degrade (e.g., use cached sensor map for discovery) rather than returning errors.
6. **Plugin Architecture:** Allow new agents to be registered via a configuration file without modifying `workflow.py`, enabling third-party extensions (e.g., a Weather Agent, an Energy Price Agent).
7. **Helm Chart:** Package the Docker Compose configuration as a Kubernetes Helm chart for cloud-native deployment with auto-scaling.
8. **Internationalisation (i18n):** Support multi-language queries and responses by adding language detection and translation capabilities to the Dialogue Agent.

This document was generated on February 18, 2026. For the latest information, refer to the repository at github.com/suhasdevmane/OntoBot.